# Lab Sheet 1: Stack Implementation Using Array

## Introduction

### Stack

A stack is a linear data structure in which insertion and deletion is performed only at one end, called the top, following the LAST IN FIRST OUT (LIFO) principle.

### Array

An array is a collection of elements of the same data type stored in consecutive memory location and accessed using an index.

### Common operations in Stack

- Push () Operation
  The push operation is used to insert new element in the stack.
  Insertion is only performed at the top of the stack. Before inserting an element, the stack is checked for overflow condition. If the stack is full, the insertion is not possible.
  Below is the example of push operation:

```
 void push (int data)
{
   if (! isFull ())
   {
      top = top + 1;
      stack[top] = data;
   }
   else
   {
      printf ("Could not insert data, Stack is full.\n");
   }
}
```

- Pop () Operation
  The pop operation is used to remove an element from the stack.
  In a stack, deletion is only allowed from the top.
  Before performing the pop operation, it is necessary to check whether the stack is empty. If the stack is empty, deletion cannot be performed. If the stack is not empty, the element at the top of the stack is removed and the top pointer is decrement by 1.
  Below is the example of the pop operation:

```
int pop ()
{
   if (! isEmpty ())
   {
```

```
        int data = stack[top];
        top = top - 1;
        return data;
    }
    else
    {
        printf ("Could not retrieve data, Stack is empty.\n");
        return -1;
    }
}
```

- isEmpty () operation
  The isEmpty operation is used to check whether the stack is empty or not.
  This operation helps to avoid stack underflow during the pop operation.
  If the value of the top pointer is equal to -1, the stack is considered empty. If the stack is empty, no deletion operations can be performed. If the stack is not empty, pop operation can be safely executed.
  Below is the example of isEmpty () operation:

```
int isEmpty ()
{
    if (top == -1)
        return 1;
    else
        return 0;
}
```

- isFull () operation
  The isFull operation is used to check whether the stack is full or not.
  This operation helps to prevent stack overflow during the push operation.
  If the value of top pointer is equal to MAX - 1, where MAX represents the maximum size of the stack, the stack is considered full. If the stack is full, no further insertion can be performed. If the stack is not full, push operation can be safely executed.
  Below is the example of isFull () operation:

```
int isFull ()
{
    if (top == MAX - 1)
        return 1;
    else
        return 0;
}
```

- peek () operation
  The peek operation is used to view the top element of the stack without removing it.

This operation allows access to the topmost element while maintaining the current state of the stack.
Before performing the peek operation, it is necessary to check whether the stack is empty or not. If the stack is empty, no element can be accessed. If the stack is not empty, the value stored at the top position is returned without modifying the top pointer.
Below is the example of peek () operation:

```
int peek ()
{
   if (! isEmpty ())
   {
      return stack[top];
   }
   else
   {
      printf ("Stack is empty.\n");
      return -1;
   }
}
```

- count () operation
  The count operation is used to determine the total number of elements available on the stack.
  This operation helps in knowing the current size of the stack at any given time.
  If the stack is empty, the count of the elements is zero. If the stack is not empty, the number of elements in the stack is calculated using the top pointer. Since, the top pointer always points to the last inserted element, the total number of elements in the stack is given by top + 1.
  Below is the example of count () operation:

```
int count ()
{
   return top + 1;
}
```

- change () operation
  The change operation is used to modify the value of an element at a specific position in the stack.
  This operation allows updating the content of the stack without affecting its structure or the top pointer.
  Before performing the change operation, it is necessary to check whether the stack is empty. If the stack is empty, no modification can be made. If the stack is not empty, the element at the specified position (from the bottom of the stack) is updated with the new value.
  Below is the example of change () operation:

```
void change (int position, int data)
{
   if (! isEmpty () && position >= 1 && position <= top + 1)
   {
      stack [position - 1] = data;
      printf ("Value at position %d changed to %d.\n", position, data);
   }
   else
   {
      printf ("Invalid position or stack is empty.\n");
   }
}
```

- display () operation
  The display operation is used to show all the elements present in the stack from top to bottom.
  This operation allows the user to view the current contents of the stack without modifying it.
  Before performing the display operation, it is necessary to check whether the stack is empty, no elements are displayed. If the stack is not empty, the elements are printed starting from top down to the bottom.
  Below is the example of display () operation:

```
void display ()
{
   if (! isEmpty ())
   {
      printf ("Stack elements are:\n");
      for (int i = top; i >= 0; i--)
      {
         printf ("%d\n", stack[i]);
      }
   }
   else
   {
      printf ("Stack is empty.\n");
   }
}
```

## Stack Conditions

- Stack Overflow
  Stack overflow occurs when an attempt is made to push (insert) an element into a stack that is already full.
  Since the stack has a fixed maximum size, no further elements can be added beyond this limit. Overflow is an error condition that must be checked to prevent program crashes.

Example:
If MAX = 5 and the stack already has 5 elements, trying to push another element will cause stack overflow.

- Stack Underflow
  Stack underflow occurs when an attempt is made to pop (remove) an element from an empty stack.
  Since there are no elements in the stack, deletion cannot be performed. Underflow is also an error condition that must be handled in programs.
  Example:
  If the stack is empty (top = -1) and we try to remove an element, it will cause stack underflow.

## Algorithm: Stack implementation using array.

Step 1: Start.

Step 2: Initialize Stack.

i. Define a constant MAX=5.
ii. Declare an integer array stack [MAX]
iii. Initialize top = -1 to indicate the stack is empty.

Step 3: isFull Operation

i. If top is greater than or equals to MAX-1, return 1 (stack is full).
ii. Else, return 0.

Step 4: isEmpty Operation

i. If top == -1 ; return 1 (stack is empty).
ii. Else return 0;

Step 5: Push operation

i. Call isFull()
ii. If the stack is full, display “Stack Overflow”
iii. Else
    - Increment top by 1
    - Insert the value into stack[top]
    - Display the success message.

Step 6: Pop Operation

i. Call isEmpty().
ii. If the stack is empty, display “Stack Underflow”
iii. Else
    - Display the element at stack[top]
    - Remove the element
    - Decrement top by 1.

Step 7: Peek Operation

i. Display the element present at stack[top].

Step 8: Display Operation

i. Call isEmpty()
ii. If the stack is empty, display “Stack is empty”
iii. Else
    - Scan the stack from top to 0.
    - Display all elements in Last In First Out order.

Step 9: Count Operation

i. Calculate the total elements using total = top +1.
ii. Display the total number of elements
iii. Return total.

Step 10: Main Execution

i. Push elements into the stack.
ii. Handle overflow condition during insertion.
iii. Perform count and peek operation.
iv. Pop elements from the stack.
v. Handle underflow conditions during deletion.
vi. Display remaining stack elements.

Step 11: Stop.

Program for stack implementation using array:

```
#include<stdio.h>
#define MAX 5

int stack[MAX];
int top=-1;

int isFull()
{
   if(top>=MAX-1)
   {
      return 1;
   }
   else
   {
      return 0;
   }
}
int isEmpty()
{
   if(top==-1)
   {
      return 1;
   }
   else
   {
      return 0;
   }
}
```

```
void push(int value)
{
   if(isFull())
   {
      printf("Stack Overflow! Cannot push %d\n", value);

   }
   else
   {
      top++;
      stack[top]=value;
      printf("%d pushed to stack\n", value);

   }
}

void pop()
{
   if(isEmpty())
   {
      printf("Stack Underflow! The stack is empty\n");

   }
   else
   {
      printf("Popped element: %d\n", stack[top]);
      stack[top]=0;
      top--;

   }
```

```
}

void peek()
{
   printf("Peeked element is: %d\n ", stack[top]);
}

void display()
{
    if (! isEmpty ())
   {
      printf ("Stack elements are:\n");
      for (int i = top; i >= 0; i--)
      {
         printf ("%d\n", stack[i]);
      }
   }
   else
   {
      printf ("Stack is empty.Stack underflow occurred!!!\n");
   }
}

int count ()
{
   int  total = top+ 1;
    printf("Total Elements currently in stack are:%d\n",total);
   return total;
}
```

```
int main()
{
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    push(60);
    count();
    peek();
    pop();
    count();
    pop();
    pop();
    display();
    pop();
    pop();
    pop();
    display();
    return 0;
}
```

Output of the given program:

```
10 pushed to stack
20 pushed to stack
30 pushed to stack
40 pushed to stack
50 pushed to stack
Stack Overflow! Cannot push 60
Total Elements currently in stack are:5
Peeked element is: 50
 Popped element: 50
Total Elements currently in stack are:4
Popped element: 40
Popped element: 30
Stack elements are:
20
10
Popped element: 20
Popped element: 10
Stack Underflow! The stack is empty
Stack is empty.Stack underflow occurred!!!
```

## Discussion :

In this program, a stack is implemented using an array in C, following the Last In First Out (LIFO) principle. The program provides basic stack operations such as push, pop, peek, isEmpty, and size through a menu-driven approach. The variable top is used to keep track of the current top element of the stack. Proper conditions are checked to handle stack overflow (when the stack is full) and stack underflow (when the stack is empty). This implementation helps in understanding how stack operations are performed using arrays and how memory is managed using a fixed-size structure.

## Conclusion :

The stack using array has been successfully implemented in C. The program correctly performs all fundamental stack operations while maintaining the LIFO order. This implementation is simple and efficient for small data sizes and is useful for understanding core stack concepts. However, since the array size is fixed, it has a limitation on memory usage. Despite this, the program serves as a good foundation for learning stack data structures and can be extended using dynamic memory or linked lists.